

econcomplex

Documentação Técnica — Versão 0.1.0

Biblioteca Python para Indicadores de
Complexidade Econômica e Ciência Regional

Elton Freitas

Dependências: `numpy` ≥ 1.21 , `pandas` ≥ 1.3 , `scipy` ≥ 1.7
Compatível com Python 3.9+

Março de 2026

Sumário

1	Visão Geral	3
1.1	Instalação	3
1.2	Estrutura de Módulos	3
2	Organização dos Dados	4
2.1	Formato Padrão: Tabela Longa (<i>long format</i>)	4
2.2	Regras de Qualidade dos Dados	4
2.3	Nomes das Colunas — Dicionário <code>cols</code>	4
2.4	Formatos de Arquivo Suportados	5
2.5	Exemplos de Bases Compatíveis	6
3	Como Executar os Scripts	6
3.1	Pré-requisitos	6
3.2	Execução Direta via Terminal	6
3.3	Execução em Jupyter Notebook / JupyterLab	7
3.4	Execução no VS Code / PyCharm / Spyder	8
3.5	Script Mínimo de Execução	8
3.6	Execução com Dados da RAIS / CNAE (caso de uso real)	9
3.7	Execução em Painel Multitemporal	9
4	Indicadores do Núcleo (<code>core</code>)	10
4.1	Vantagem Comparativa Revelada (RCA)	10
4.2	Diversidade e Ubiquidade	10
5	Complexidade Econômica (<code>complexity</code>)	11
5.1	Método do Vetor Próprio (Hidalgo & Hausmann, 2009)	11
5.2	Método de Reflexões (Hidalgo & Hausmann, 2009)	11
5.3	Fitness-Complexity (Tacchini et al., 2012)	11
5.4	ECI Subnacional	11
6	Product Space e Relatedness (<code>relatedness</code>)	11
6.1	Proximidade (ϕ)	11
6.2	Densidade e Distância	12
6.3	Co-ocorrência e Índices de Relatedness	12
6.4	Cross-Space (relatedness entre classificações distintas)	12
7	Especialização Regional (<code>specialization</code>)	12
8	Desigualdade e Concentração (<code>inequality</code>)	12
9	Produtividade (<code>productivity</code>)	13
10	Complexidade Baseada em Patentes (<code>patents</code>)	13
11	Dinâmica Temporal (<code>dynamics</code>)	13
12	Complexity Outlook (<code>outlook</code>)	13

13 Pipeline Unificado (<code>compute_complexity</code>)	13
13.1 Assinatura da Função	13
13.2 Parâmetros	14
14 Exemplos Completos	14
14.1 Exemplo 1 — Análise Regional de Empregos	14
14.2 Exemplo 2 — Exportações com Dados COMEX	15
14.3 Exemplo 3 — Painel Temporal RAIS (2010–2022)	15
15 Referências Bibliográficas	16
16 Licença e Citação	16

Visão Geral

econcomplex é uma biblioteca Python que consolida as melhores implementações de indicadores de Complexidade Econômica e de Ciência Regional, reunindo funcionalidades de quatro pacotes de referência na literatura:

- **EconGeo** (Balland, 2017) — pacote R com foco em indicadores geográficos e de especialização regional
- **economiccomplexity** (Vargas Sepúlveda, 2020) — pacote R com backend C++ (Armadillo) para indicadores de complexidade
- **py-econcomplexity** (The Growth Lab at Harvard, 2020) — pipeline completo em Python com suporte a séries temporais
- **py-economic-complexity** — implementação modular em Python com suporte a Polars e indicadores de cross-space

Instalação

```

1 # A partir do diretório raiz da biblioteca
2 pip install -e .
3
4 # Ou adicionando o caminho manualmente
5 import sys
6 sys.path.insert(0, "/caminho/para/econcomplex")
7 import econcomplex as ec

```

Listing 1: Instalação via pip (desenvolvimento local)

Estrutura de Módulos

Módulo	Conteúdo
core	RCA, RPOP, Mcp, diversidade, ubiquidade, utilitários
complexity	ECI/PCI (3 métodos), ECI subnacional
relatedness	Proximidade, densidade, distância, co-ocorrência, cross-space
specialization	LQ, Hachman, Krugman, coef. especialização, similaridade
inequality	Gini, Gini locacional, Hoover-Gini, HHI, entropia Shannon
productivity	PRODY, EXPY, PGI, PEII
patents	Ease of Recombination, Modular Complexity
dynamics	Taxas de crescimento, rastreamento de entrada/saída
outlook	COI, COG (Complexity Outlook)
pipeline	Função <code>compute_complexity()</code> — pipeline unificado

Organização dos Dados

Esta seção descreve o formato de entrada esperado pela biblioteca, garantindo que os dados estejam corretamente estruturados antes da execução de qualquer indicador.

Formato Padrão: Tabela Longa (*long format*)

A biblioteca **econcomplex** trabalha exclusivamente com **tabelas no formato longo** (*tidy data*), onde cada linha representa **uma combinação única** de região × atividade (e período, quando houver):

regiao	atividade	valor	ano (opcional)
SP	cnae_10	12 345	2022
SP	cnae_25	6 789	2022
RJ	cnae_10	9 012	2022
RJ	cnae_25	3 456	2022
SP	cnae_10	13 200	2023
⋮	⋮	⋮	⋮

Atenção

A biblioteca **não** aceita matrizes pivotadas (regiões nas linhas, atividades nas colunas). Use `pivot_to_matrix()` apenas para funções de baixo nível que exijam a matriz explicitamente.

Regras de Qualidade dos Dados

1. **Sem linhas duplicadas:** cada par (*regiao*, *atividade*) deve aparecer exatamente uma vez por período.
2. **Valores não-negativos:** a coluna de valor deve conter apenas números ≥ 0 .
3. **Sem NaN:** remova ou impute valores ausentes antes de passar o DataFrame.
4. **Granularidade consistente:** todas as regiões devem usar o mesmo nível geográfico (ex.: todos municípios *ou* todos estados).
5. **Atividades homogêneas:** use uma única classificação por análise (ex.: CNAE 2 dígitos *ou* CNAE 4 dígitos, não misturados).

Nomes das Colunas — Dicionário *cols*

O pipeline usa um dicionário *cols* para mapear os nomes reais das colunas do seu DataFrame:

Chave	Tipo	Descrição
"loc"	obrigatória	Coluna de região / localização
"act"	obrigatória	Coluna de atividade / produto / tecnologia
"val"	obrigatória	Coluna de valor numérico (empregos, exportações ...)
"time"	opcional	Coluna de período (ano, trimestre ...)

```

1 import pandas as pd
2 import econcomplex as ec
3
4 df = pd.read_csv("meus_dados.csv")
5
6 # Verificar estrutura
7 print(df.dtypes)
8 print(df.shape)
9 print(df.isnull().sum())      # checar NaN
10 print(df.duplicated().sum())  # checar duplicatas
11
12 # Limpeza basica
13 df = df.dropna(subset=["regiao", "cnae", "empregos"])
14 df = df[df["empregos"] >= 0]  # remover valores negativos
15 df = df.drop_duplicates(subset=["regiao", "cnae", "ano"])
16
17 # Rodar o pipeline
18 resultado = ec.compute_complexity(
19     df,
20     cols={"loc": "regiao", "act": "cnae",
21           "val": "empregos", "time": "ano"},
22 )

```

Listing 2: Verificação e preparação do DataFrame antes do pipeline

Formatos de Arquivo Suportados

Formato	Leitura pandas	Observação
.csv	pd.read_csv()	Mais comum; use sep=";" para CSVs BR
.parquet	pd.read_parquet()	Ideal para bases grandes (RAIS, COMEX)
.xlsx	pd.read_excel()	Requer openpyxl; evitar >100 000 linhas
.feather	pd.read_feather()	Alta performance; requer pyarrow
.dta	pd.read_stata()	Arquivos Stata; preserva labels

```

1 import pandas as pd
2
3 # CSV com separador ponto-e-virgula (padrao brasileiro)
4 df = pd.read_csv("rais.csv", sep=";", encoding="latin-1",
5                 usecols=["municipio", "cnae2d", "ano", "vinculos"],
6                 dtype={"municipio": str, "cnae2d": str})
7
8 # Parquet (preferencial para RAIS/COMEX)
9 df = pd.read_parquet("comex_ncm.parquet",
10                    columns=["co_mun", "co_ncm", "ano", "vl_fob"])
11
12 # Excel
13 df = pd.read_excel("dados.xlsx", sheet_name="Sheet1",
14                   usecols=["uf", "setor", "ano", "empregos"])

```

Listing 3: Leitura de diferentes formatos de arquivo

Exemplos de Bases Compatíveis

RAIS/MTE

Empregos formais por município × CNAE (2 ou 4 dígitos) × ano. Coluna de valor: vínculos ativos em 31/12.

COMEX/MDIC

Exportações por município × NCM × ano. Coluna de valor: VL_FOB (valor em US\$).

INPI — Patentes

Pedidos de patente por Unidade da Federação × classificação IPC × ano.

Atlas da Complexidade (Harvard)

Exportações por país × produto (HS) × ano. Disponível em <https://atlas.cid.harvard.edu/>.

Dados simulados (testes)

Use `ec.make_sample_data(n_locs=20, n_acts=50, seed=42)` para gerar um DataFrame sintético.

Como Executar os Scripts

Esta seção descreve todas as formas de executar a biblioteca **econcomplex**, desde a execução direta de scripts Python até o uso em ambientes interativos como Jupyter e IDEs.

Pré-requisitos

Atenção

Certifique-se de que as dependências estão instaladas antes de executar qualquer script.

```
1 # Instalar dependencias principais
2 pip install numpy pandas scipy
3
4 # Instalar a biblioteca em modo de desenvolvimento
5 pip install -e /caminho/para/econcomplex
6
7 # Verificar instalacao
8 python3 -c "import econcomplex as ec; print(ec.__version__)"
```

Listing 4: Instalação das dependências

Execução Direta via Terminal

A forma mais simples de usar a biblioteca é executar um script `.py` diretamente no terminal:

```
1 # Navegue ate a pasta do projeto
2 cd "/caminho/para/1 - Pos-doc research/data/script"
3
4 # Executar o script de exemplo completo
5 python3 econcomplex/exemplo_uso.py
6
7 # Executar um script proprio
8 python3 meu_script.py
```

Listing 5: Execução de scripts no terminal

Caso a biblioteca **não** esteja instalada via **pip**, adicione o caminho manualmente no início de cada script:

```
1 import sys
2 sys.path.insert(0, "/caminho/para/econcomplex")
3
4 import econcomplex as ec
5 print(ec.__version__) # 0.1.0
```

Listing 6: Adicionar caminho ao sys.path (sem pip install)

Execução em Jupyter Notebook / JupyterLab

```
1 cd "/caminho/para/1 - Pos-doc research/data/script"
2 jupyter notebook
3 # ou
4 jupyter lab
```

Listing 7: Abrir Jupyter a partir da pasta de scripts

Dentro do notebook, a primeira célula deve configurar o caminho:

```
1 # Celula 1 -- configuracao
2 import sys, os
3
4 BASE = os.path.expanduser(
5     "~/Library/CloudStorage/"
6     "OneDrive-Personnel/1 - Pos-doc research/data/script/econcomplex"
7 )
8 sys.path.insert(0, BASE)
9
10 import econcomplex as ec
11 import numpy as np
12 import pandas as pd
13
14 print("econcomplex", ec.__version__, "carregado com sucesso.")
```

Listing 8: Célula de configuração no Jupyter

```
1 # Celula 2 -- carregar dados
2 df = pd.read_csv("meus_dados.csv") # colunas: regioao, setor, empregos
3
4 # Celula 3 -- calcular complexidade
5 resultado = ec.compute_complexity(
6     df,
7     cols={"loc": "regiao", "act": "setor", "val": "empregos"},
8     method="eigenvector",
9     compute_coi_cog=True,
10 )
11 resultado.head()
12
13 # Celula 4 -- visualizar ECI por regioao
14 eci = (resultado[["regiao", "eci"]]
15        .drop_duplicates()
16        .set_index("regiao")
17        .sort_values("eci", ascending=False))
18 print(eci)
```

Listing 9: Células subsequentes — uso normal

Execução no VS Code / PyCharm / Spyder

Em qualquer IDE, configure o **interpretador Python** e o **diretório de trabalho** apontando para a pasta de scripts.

```
1 // .vscode/settings.json
2 {
3     "python.defaultInterpreterPath":
4         "/Library/Frameworks/Python.framework/Versions/3.12/bin/python3",
5     "terminal.integrated.cwd": "${workspaceFolder}"
6 }
```

Listing 10: Configurar caminho no VS Code (settings.json)

No **Spyder**, vá em *Tools* → *PYTHONPATH Manager* e adicione o diretório `econcomplex/`.

Script Mínimo de Execução

Template pronto para uso com qualquer base em formato CSV:

```
1 """
2 Template de uso da biblioteca econcomplex.
3 Adapte os nomes das colunas e o caminho do arquivo CSV.
4 """
5 import sys
6 sys.path.insert(0, "/caminho/para/econcomplex")
7
8 import pandas as pd
9 import econcomplex as ec
10
11 # 1. Carregar dados
12 df = pd.read_csv("dados.csv")
13 print("Colunas:", df.columns.tolist())
14 print("Shape:", df.shape)
15
16 # 2. Calcular todos os indicadores
17 resultado = ec.compute_complexity(
18     df,
19     cols={
20         "loc": "nome_coluna_regiao",      # ex: "municipio", "uf"
21         "act": "nome_coluna_atividade",   # ex: "cnae", "produto"
22         "val": "nome_coluna_valor",       # ex: "empregos", "exportacoes"
23         "time": "nome_coluna_ano",        # opcional
24     },
25     method="eigenvector",               # 'eigenvector' | 'reflections' | 'fitness'
26     threshold=1.0,
27     compute_coi_cog=True,
28 )
29
30 # 3. Exportar resultados
31 resultado.to_csv("resultados_complexidade.csv", index=False)
32 print("Resultado salvo em resultados_complexidade.csv")
33 print("Colunas geradas:", sorted(resultado.columns.tolist()))
34
35 # 4. Indicadores individuais (opcional)
36 mat = ec.pivot_to_matrix(df,
37                             "nome_coluna_regiao",
38                             "nome_coluna_atividade",
```

```

39         "nome_coluna_valor")
40
41 eci, pci = ec.eci_pci(mat)
42 hachman = ec.hachman_index(mat)
43 krugman = ec.krugman_index(mat)
44 entropia = ec.shannon_entropy(mat)
45
46 saida = pd.DataFrame({
47     "eci": eci, "hachman": hachman,
48     "krugman": krugman, "entropia": entropia,
49 })
50 print(saida.round(3))

```

Listing 11: Template mínimo — copiar e adaptar

Execução com Dados da RAIS / CNAE (caso de uso real)

```

1 import sys
2 sys.path.insert(0, "/caminho/para/econcomplex")
3
4 import pandas as pd
5 import econcomplex as ec
6
7 # Carregar RAIS agregada
8 # Esperado: municipio, cnae2d, ano, vinculos
9 rais = pd.read_parquet("rais_municipio_cnae.parquet")
10
11 # Filtrar um unico ano para analise
12 rais_2022 = rais[rais["ano"] == 2022].copy()
13
14 # Pipeline
15 resultado = ec.compute_complexity(
16     rais_2022,
17     cols={"loc": "municipio", "act": "cnae2d", "val": "vinculos"},
18     method="eigenvector",
19     compute_coi_cog=False, # desligar COG para bases grandes
20 )
21
22 # Top 20 municipios mais complexos
23 top20 = (resultado[["municipio", "eci"]]
24     .drop_duplicates()
25     .sort_values("eci", ascending=False)
26     .head(20))
27 print(top20.to_string(index=False))
28
29 resultado.to_csv("complexidade_municipios_2022.csv", index=False)

```

Listing 12: Uso com dados RAIS — municípios x CNAE

Execução em Painel Multitemporal

Para analisar a evolução dos indicadores ao longo do tempo, basta incluir a coluna temporal no dicionário `cols`:

```

1 # rais com coluna "ano" contendo 2010, 2015, 2020
2 resultado_painel = ec.compute_complexity(
3     rais,

```

```

4     cols={
5         "loc": "municipio",
6         "act": "cnae2d",
7         "val": "vinculos",
8         "time": "ano",          # ativa o loop temporal
9     },
10    method="reflections",
11    compute_coi_cog=False,
12 )
13
14 # O resultado ja contem todos os anos
15 print(resultado_painel["ano"].value_counts().sort_index())
16
17 # Evolucao do ECI mediano por ano
18 eci_evolucao = (resultado_painel[["ano", "municipio", "eci"]]
19                 .drop_duplicates()
20                 .groupby("ano")["eci"]
21                 .median()
22                 .round(4))
23 print(eci_evolucao)

```

Listing 13: Execução em painel — múltiplos anos

Nota

O pipeline itera automaticamente sobre cada período, garantindo que a matriz de proximidade e os vetores de complexidade sejam recalculados de forma independente para cada ano.

Indicadores do Núcleo (core)

Vantagem Comparativa Revelada (RCA)

A RCA de Balassa é o principal filtro de especialização:

$$RCA_{il} = \frac{x_{il} / \sum_l x_{il}}{\sum_i x_{il} / \sum_{il} x_{il}}$$

```

1 import econcomplex as ec
2 import pandas as pd
3
4 df = pd.read_csv("dados.csv")
5 mat = ec.pivot_to_matrix(df, "regiao", "setor", "empregos")
6
7 rca      = ec.rca(mat)          # valores continuos
8 mcp      = ec.mcp(mat, threshold=1) # 1 se RCA >= 1, 0 caso contrario
9 rca_rpop = ec.rpop(mat)        # RCA de Puga & Turrini (continuo)

```

Listing 14: RCA contínuo e binário

Diversidade e Ubiquidade

```

1 div = ec.diversity(mat) # numero de setores com RCA >= 1 por regiao
2 ubi = ec.ubiquity(mat) # numero de regioes com RCA >= 1 por setor

```

Listing 15: Diversidade e ubiquidade

Complexidade Econômica (complexity)

Método do Vetor Próprio (Hidalgo & Hausmann, 2009)

$$ECI = \frac{\vec{v}_2 - \mu(\vec{v}_2)}{\sigma(\vec{v}_2)}, \quad PCI = \frac{\vec{u}_2 - \mu(\vec{u}_2)}{\sigma(\vec{u}_2)}$$

onde $\vec{v}_2$ é o segundo autovetor esquerdo da matriz $\tilde{M}$:

$$\tilde{M}_{ij} = \frac{M_{ij}}{k_{c,0} \cdot k_{p,0}}$$

```
1 eci, pci = ec.eci_pci(mat, method="eigenvector")
2 print(eci.head()) # Series indexada por regioao
3 print(pci.head()) # Series indexada por produto/setor
```

Listing 16: ECI e PCI pelo método do vetor próprio

Método de Reflexões (Hidalgo & Hausmann, 2009)

```
1 eci_r, pci_r = ec.eci_pci(mat, method="reflections", iterations=20)
```

Listing 17: Método de Reflexões

Fitness-Complexity (Tacchini et al., 2012)

$$F_c^{(n+1)} = \sum_p M_{cp} Q_p^{(n)}, \quad Q_p^{(n+1)} = \frac{1}{\sum_c M_{cp} / F_c^{(n)}}$$

```
1 fit, comp = ec.eci_pci(mat, method="fitness", iterations=100)
```

Listing 18: Fitness-Complexity

ECI Subnacional

```
1 # pci_nacional: PCI calculado com dados nacionais/internacionais
2 eci_sub = ec.subnational_eci(mat_sub, pci_nacional)
```

Listing 19: ECI subnacional com PCI externo

Product Space e Relatedness (relatedness)

Proximidade (ϕ)

$$\phi_{ij} = \min(P(RCA_i \geq 1 \mid RCA_j \geq 1), P(RCA_j \geq 1 \mid RCA_i \geq 1))$$

```
1 phi_d = ec.proximity(mat, continuous=False) # baseada em Mcp binario
2 phi_c = ec.proximity(mat, continuous=True)  # baseada em RCA contínuo
```

Listing 20: Proximidade discreta e contínua

Densidade e Distância

$$\omega_{ij} = \frac{\sum_k M_{kj} \phi_{ij}}{\sum_k \phi_{ij}}, \quad d_{ij} = 1 - \omega_{ij}$$

```
1 density = ec.density(mat, phi_d) # DataFrame loc x act
2 distance = ec.distance(mat, phi_d) # DataFrame loc x act
```

Listing 21: Densidade e distância

Co-ocorrência e Índices de Relatedness

```
1 cooc = ec.co_occurrence(mat)
2 phi_cos = ec.cosine_proximity(mat) # baseada em cosseno
3 phi_cor = ec.correlation_proximity(mat)
4 rca_rel = ec.relatedness(mat, phi_d) # densidades de relatedness
```

Listing 22: Co-ocorrência e índices de relatedness

Cross-Space (relatedness entre classificações distintas)

```
1 # phi_cross: proximidade entre setores e tecnologias (matrizes
   # diferentes)
2 phi_cross = ec.cross_space_proximity(mat_setor, mat_tech)
3 density_c = ec.density(mat_setor, phi_cross)
```

Listing 23: Proximidade e relatedness cross-space

Especialização Regional (specialization)

```
1 lq = ec.location_quotient(mat) # equivalente ao RCA
2 hachman = ec.hachman_index(mat) # 0 (diversificado) a 1
   # (especializado)
3 krugman = ec.krugman_index(mat) # diferenca estrutural entre
   # regioes
4 coef_esp = ec.spec_coefficient(mat) # coeficiente de especializacao
5 sim_exp = ec.export_similarity(mat) # similaridade de estrutura
   # produtiva
```

Listing 24: Índices de especialização

Desigualdade e Concentração (inequality)

```
1 gini_r = ec.gini(mat) # Gini da distribuicao do valor
2 gini_loc = ec.locational_gini(mat) # Gini locacional (por setor)
3 hoover = ec.hoover_gini(mat) # Indice de Hoover
4 hhi = ec.hhi(mat) # Herfindahl-Hirschman
5 entropia = ec.shannon_entropy(mat) # entropia de Shannon (por regioao)
```

Listing 25: Indicadores de desigualdade

Produtividade (productivity)

```

1 prody = ec.prody(mat, gdp_per_capita) # renda associada ao produto
2 expy  = ec.expy(mat, prody)          # sofisticacao da cesta
    exportadora
3 pgi   = ec.pgi(mat)                  # Product Gini Index
4 peii  = ec.peii(mat)                 # Product Export Inequality Index

```

Listing 26: Produtividade e desigualdade dos produtos

Complexidade Baseada em Patentes (patents)

```

1 ease_rec = ec.ease_of_recombination(mat_pat)
2 mod_comp = ec.modular_complexity(mat_pat)

```

Listing 27: Indicadores de patentes (input: matriz patente x tecnologia)

Dinâmica Temporal (dynamics)

```

1 growth = ec.growth_rates(df, "regiao", "setor", "empregos", "ano")
2 entries = ec.entry_tracking(df, "regiao", "setor", "ano")
3 exits   = ec.exit_tracking(df, "regiao", "setor", "ano")

```

Listing 28: Crescimento e rastreamento de entrada/saída

Complexity Outlook (outlook)

O **COI** mede o potencial latente de diversificação; o **COG** mede o ganho esperado de complexidade:

$$COI_c = \sum_p (1 - M_{cp}) \omega_{cp} Q_p$$

```

1 coi = ec.coi(mat, phi_d, pci)
2 cog = ec.cog(mat, phi_d, pci)

```

Listing 29: COI e COG

Pipeline Unificado (compute_complexity)

Assinatura da Função

```

1 def compute_complexity(
2     df: pd.DataFrame,
3     cols: dict,          # {"loc": str, "act": str, "val": str, "time":
        str}
4     method: str = "eigenvector", # 'eigenvector' / 'reflections' /
        'fitness'
5     threshold: float = 1.0,      # limiar para Mcp
6     compute_coi_cog: bool = True,
7     iterations: int = 20,        # para reflections/fitness
8 ) -> pd.DataFrame:

```

```

9      """
10     Retorna o DataFrame original enriquecido com as colunas:
11     rca, mcp, diversity, ubiquity, eci, pci,
12     density, distance, [coi, cog]
13     """

```

Listing 30: Assinatura da função pipeline

Parâmetros

Parâmetro	Tipo	Descrição
df	DataFrame	Tabela no formato longo
cols	dict	Mapeamento de colunas (ver Seção 2.3)
method	str	Método de complexidade
threshold	float	Limiar do Mcp (padrão: 1.0)
compute_coi_cog	bool	Calcular COI e COG
iterations	int	Iterações para métodos iterativos

Exemplos Completos

Exemplo 1 — Análise Regional de Empregos

```

1  import sys
2  sys.path.insert(0, "/caminho/para/econcomplex")
3
4  import pandas as pd
5  import numpy as np
6  import econcomplex as ec
7
8  # Gerar dados sinteticos (ou carregar CSV real)
9  df = ec.make_sample_data(n_locs=50, n_acts=30, seed=42)
10 # Colunas: loc, act, val
11
12 # Pipeline completo
13 resultado = ec.compute_complexity(
14     df,
15     cols={"loc": "loc", "act": "act", "val": "val"},
16     method="eigenvector",
17     compute_coi_cog=True,
18 )
19
20 # Resumo dos indicadores gerados
21 print(resultado.columns.tolist())
22 # ['loc', 'act', 'val', 'rca', 'mcp', 'diversity', 'ubiquity',
23 #  'eci', 'pci', 'density', 'distance', 'coi', 'cog']
24
25 # Top 10 regioes por ECI
26 top_eci = (resultado[["loc", "eci"]]
27             .drop_duplicates()
28             .sort_values("eci", ascending=False)
29             .head(10))

```

```
30 print(top_eci)
31
32 # Salvar
33 resultado.to_csv("resultado_regional.csv", index=False)
```

Listing 31: Análise completa de emprego regional

Exemplo 2 — Exportações com Dados COMEX

```
1 import sys
2 sys.path.insert(0, "/caminho/para/econcomplex")
3
4 import pandas as pd
5 import econcomplex as ec
6
7 # Carregar base COMEX agregada por municipio x NCM x ano
8 comex = pd.read_parquet("comex_mun_ncm.parquet")
9 comex_2023 = comex[comex["ano"] == 2023].copy()
10
11 # Limpeza
12 comex_2023 = comex_2023.dropna(subset=["co_mun", "co_ncm", "vl_fob"])
13 comex_2023 = comex_2023[comex_2023["vl_fob"] > 0]
14
15 # Pipeline
16 resultado = ec.compute_complexity(
17     comex_2023,
18     cols={"loc": "co_mun", "act": "co_ncm", "val": "vl_fob"},
19     method="eigenvector",
20     compute_coi_cog=True,
21 )
22
23 # Salvar
24 resultado.to_csv("complexidade_exportacoes_2023.csv", index=False)
25 print("ECI medio:", resultado["eci"].dropna().mean().round(4))
```

Listing 32: Análise de exportacoes com dados COMEX/MDIC

Exemplo 3 — Painel Temporal RAIS (2010–2022)

```
1 import sys
2 sys.path.insert(0, "/caminho/para/econcomplex")
3
4 import pandas as pd
5 import econcomplex as ec
6
7 rais = pd.read_parquet("rais_municipio_cnae_2010_2022.parquet")
8
9 # Pipeline temporal -- itera sobre cada ano automaticamente
10 painel = ec.compute_complexity(
11     rais,
12     cols={
13         "loc": "municipio",
14         "act": "cnae2d",
15         "val": "vinculos",
16         "time": "ano",
17     },
18     method="eigenvector",
```

```
19     compute_coi_cog=False,
20 )
21
22 # Evolucao do ECI mediano
23 eci_trend = (painel[["ano", "municipio", "eci"]]
24               .drop_duplicates()
25               .groupby("ano")["eci"]
26               .median()
27               .round(4))
28 print(eci_trend)
29
30 painel.to_csv("painel_complexidade_2010_2022.csv", index=False)
```

Listing 33: Pipeline com dados em painel — multiplos periodos

Referências Bibliográficas

Balassa (1965)

Balassa, B. (1965). Trade liberalisation and “revealed” comparative advantage. *The Manchester School*, 33(2), 99–123.

Hidalgo & Hausmann (2009)

Hidalgo, C.A., & Hausmann, R. (2009). The building blocks of economic complexity. *PNAS*, 106(26), 10570–10575.

Balland (2017)

Balland, P.-A. (2017). *EconGeo: Computing Key Indicators of the Geography of Innovation*. R package.

Vargas Sepúlveda (2020)

Vargas Sepúlveda, M. (2020). *economiccomplexity: Computational Methods for Economic Complexity*. R package.

The Growth Lab at Harvard (2020)

The Growth Lab at Harvard University (2020). *py-econcomplexity: Economic Complexity in Python*. GitHub.

Tacchini et al. (2012)

Tacchini, E., Loreto, V., Pietronero, L. (2012). A New Metrics for Countries’ Fitness and Products’ Complexity. *Scientific Reports*, 2, 723.

Licença e Citação

Esta biblioteca é distribuída sob a licença MIT. Para uso em trabalhos acadêmicos, sugerimos citar os pacotes originais nas versões correspondentes aos indicadores utilizados.

```
1 import econcomplex as ec
2 print(ec.__version__)    # 0.1.0
```

Listing 34: Verificar versão da biblioteca